

Chicago Business Owners Pipeline Documentation

Quinten Ballard

Contents

1 Project Overview	2
2 Data Source Description	2
2.1 Business Owners Dataset	2
2.2 Business Licenses Dataset	2
3 Purpose of the Merged Dataset	3
4 Pipeline Components	4
5 Pipeline Flow	4
6 Dependencies and Prerequisites	5
6.1 System Requirements	5
6.2 Python Package Dependencies	5
6.3 Windows and WSL Notes	6
6.4 Environment Variables	6
7 Manual Pipeline Execution	6
8 Running the Pipeline with Airflow	7
9 Transformation Logic	7
9.1 Owner Data Cleaning	7
9.2 License Data Cleaning	7
9.3 Merge Logic	8
10 Output Files	8
11 Testing Strategy	8
11.1 Download Testing	8
11.2 Transformation Testing	9
11.3 Validation Testing	9
11.4 Reliability Testing	10
11.5 Data Correctness Testing	10
12 Conclusion	10

1 Project Overview

The Chicago Business Owners pipeline is a local PySpark-based data engineering pipeline designed to download, clean, merge, validate, and store business ownership and business license data from the City of Chicago Data Portal.

The pipeline uses two public datasets:

- **Business Owners:** Contains owner-level information for business accounts listed in the City of Chicago Business Licenses dataset.
- **Business Licenses:** Contains business license records issued by the City of Chicago Department of Business Affairs and Consumer Protection from 2002 to the present.

The goal of the pipeline is to create clean Parquet outputs that can be used for analysis of Chicago business activity, ownership, licensing patterns, geography, license status, and business operations.

2 Data Source Description

2.1 Business Owners Dataset

The Business Owners dataset describes the people or legal entities associated with business license accounts in Chicago. Each record is tied to an **Account Number**, which acts as the relationship key between owner information and business license information.

Important fields include:

- **Account Number:** Business account identifier used to connect owner data to license data.
- **Legal Name:** Legal business name.
- **Owner First Name:** First name of the business owner.
- **Owner Middle Initial:** Middle initial of the business owner.
- **Owner Last Name:** Last name of the business owner.
- **Legal Entity Owner:** Legal entity that owns or controls the business, when applicable.
- **Title:** Owner role, such as president, secretary, managing member, sole proprietor, or other title.

This dataset helps answer questions about who owns or controls businesses that operate under Chicago business licenses.

2.2 Business Licenses Dataset

The Business Licenses dataset describes licenses issued to businesses by the City of Chicago. It includes license identifiers, license status, application type, business activity, address fields, geographic fields, and important license dates.

Important fields include:

- **Account Number:** Business account identifier.

- **License ID:** Unique license record identifier.
- **Application Type:** Indicates whether the record is an initial issue, renewal, change of location, change of capacity, expansion, or change of business activity.
- **License Status:** Indicates whether the license was issued, cancelled, revoked, or appealed.
- **License Term Start Date:** Start date of the license term.
- **License Term Expiration Date:** Expiration date of the license term.
- **Date Issued:** Date the license was issued.
- **Ward, Precinct, Police District, and Community Area:** Geographic and administrative fields.
- **Latitude and Longitude:** Coordinates for location-based analysis.

This dataset helps answer questions about what businesses are licensed, where they operate, what type of license activity occurred, and whether licenses are active, cancelled, revoked, or appealed.

3 Purpose of the Merged Dataset

The merged dataset combines business license records with business owner information using `account_number`. This creates a more complete view of licensed business activity in Chicago.

The Business Licenses dataset answers:

What businesses are licensed, where are they located, what license activity occurred, and what is the license status?

The Business Owners dataset answers:

Who owns or controls the business account connected to those licenses?

After merging, the combined dataset can answer stronger business questions, such as:

- Which owners are connected to multiple business licenses?
- Which legal entities operate across multiple locations?
- Which communities, wards, or police districts have the highest licensed business activity?
- Which license types or application types are most common?
- Which owners or businesses have revoked, cancelled, or appealed licenses?
- How business license activity has changed over time.
- Where specific owners, legal entities, or business names are operating in Chicago.

This data can support business compliance analysis, economic development research, geographic business analysis, ownership transparency, and operational reporting for city business activity.

4 Pipeline Components

Component	Purpose
pipeline/download_csvs.py	Downloads the Business Owners and Business Licenses CSV files from URLs stored in the <code>.env</code> file. Includes retry logic, streaming downloads, timeout handling, and temporary <code>.part</code> files to avoid keeping incomplete downloads.
pipeline/transform.py	Reads the downloaded CSV files into Spark DataFrames, cleans both datasets, merges them using <code>account_number</code> , and writes the raw, cleaned, and merged outputs as Snappy-compressed Parquet files.
pipeline/validation.py	Validates the generated Parquet outputs by checking that output folders exist, files are readable by Spark, datasets are not empty, required columns are present, and important fields do not contain null values.
pipeline/clean_up.py	Deletes downloaded CSV files and leftover partial download files after validation succeeds. This prevents large raw CSV files from remaining in the project directory after successful processing.
dags/pipeline_dag.py	Defines the Airflow DAG that orchestrates the full workflow. The DAG downloads both datasets, transforms and writes Parquet outputs, validates the outputs, and cleans up downloaded files.
parquet/	Stores generated Parquet output folders. The repository keeps this folder using a <code>.gitkeep</code> file, while generated Parquet data is created at runtime.

Table 1: Main components of the Chicago Business Owners data pipeline.

5 Pipeline Flow

The pipeline follows this task order:

```
download_business_owners
download_business_licenses
    |
    v
transform_and_write_parquet
    |
    v
validate_parquet_outputs
    |
    v
delete_downloaded_csvs
```

The two download tasks can run before the transformation step. Once both CSV files are available, the transformation task reads the data, cleans it, merges it, and writes Parquet outputs. Validation runs after transformation. Cleanup only runs after validation succeeds.

6 Dependencies and Prerequisites

6.1 System Requirements

The pipeline requires the following system-level prerequisites:

- Python 3.x
- A Python virtual environment for isolating project dependencies
- Java installed and available through `JAVA_HOME`
- PySpark
- Apache Airflow
- Git, if cloning the project from GitHub

If running this project on Windows, it is recommended to use WSL2 with a Linux distribution such as Ubuntu. This is especially important for Airflow, because Airflow is most commonly run in a Linux-like environment. The project was developed and tested from a Linux terminal environment through WSL.

Java is required because PySpark runs on the JVM. Spark can run locally as long as Java is installed on the system path or `JAVA_HOME` points to a valid Java installation. PySpark also requires Java 17 or later with `JAVA_HOME` properly configured.

6.2 Python Package Dependencies

The project uses the following Python packages:

- `requests`
- `python-dotenv`
- `pyspark`
- `apache-airflow`
- `pendulum`

The dependencies can be installed manually inside a virtual environment:

```
python -m venv ~/.venvs/chicago-airflow
source ~/.venvs/chicago-airflow/bin/activate
pip install requests python-dotenv pyspark apache-airflow pendulum
```

After installing Java, verify that Java is available:

```
java -version
echo $JAVA_HOME
```

If `JAVA_HOME` is empty, it should be set before running the PySpark transformation step.

6.3 Windows and WSL Notes

On Windows, the recommended setup is:

- Install WSL2.
- Install Ubuntu through WSL.
- Clone or open the project inside the WSL terminal.
- Create and activate a Python virtual environment.
- Install the required Python packages.
- Install Java inside WSL.
- Run Airflow, PySpark, and pipeline commands from the WSL terminal.

Running Airflow directly from native Windows is not recommended for this project. WSL provides a more consistent Linux-like environment for Airflow, Spark, file paths, shell commands, and scheduling behavior.

6.4 Environment Variables

The download module expects a `.env` file in the project root containing the dataset CSV URLs.

Example:

```
BUSINESSOWNERSCSVURL=https://data.cityofchicago.org/resource/ezma-pppn.csv
BUSINESSLICENSESCSVURL=https://data.cityofchicago.org/resource/r5kz-chrr.csv
```

The pipeline reads these values using `python-dotenv`. If either value is missing, the download task raises a runtime error.

7 Manual Pipeline Execution

The pipeline can be run manually without Airflow by executing the major stages in order.

From the project root, run:

```
python -c "from pipeline.download_csvs import download_business_owners,
          download_business_licenses; download_business_owners(); download_business_licenses()"
```

Then run the transformation step:

```
python -c "from pipeline.transform import transform_and_write_parquet;
          transform_and_write_parquet('business_owners.csv', 'business_licenses.csv')"
```

Then run validation:

```
python -c "from pipeline.validation import validate_parquet_outputs;
          validate_parquet_outputs()"
```

Finally, if validation succeeds, run cleanup:

```
python -c "from pipeline.clean_up import delete_downloaded_csvs; delete_downloaded_csvs('
          business_owners.csv', 'business_licenses.csv')"
```

The cleanup step should only be run after validation succeeds, because it deletes the downloaded CSV files.

8 Running the Pipeline with Airflow

The Airflow DAG is defined in:

```
dags/pipeline_dag.py
```

The DAG ID is:

```
pipeline_dag
```

The DAG is scheduled with the cron expression:

```
0 0 * * *
```

This means the pipeline is scheduled to run every day at midnight.

The DAG settings include:

- `catchup=False`: Airflow will not automatically backfill missed historical runs.
- `max_active_runs=1`: Only one DAG run can be active at a time.
- `Timezone: America/New_York`

To run through Airflow, start the Airflow scheduler and webserver, then enable and trigger the DAG from the Airflow UI.

9 Transformation Logic

9.1 Owner Data Cleaning

The owner cleaning step performs the following operations:

- Casts `Account Number` to string.
- Trims whitespace from owner name fields.
- Converts owner first name, middle initial, and last name fields to title case.
- Creates a new `full_name` field from owner name columns.
- Trims all string columns.
- Standardizes column names by stripping spaces, converting to lowercase, and replacing spaces with underscores.
- Removes duplicate rows.

9.2 License Data Cleaning

The license cleaning step performs the following operations:

- Trims all string columns.
- Standardizes column names.

- Converts date fields using `try_to_date`.
- Converts integer-like fields such as `license_id`, `ward`, `precinct`, `police_district`, and `community_area`.
- Converts coordinate fields such as `latitude` and `longitude` to double.
- Filters out records where `account_number` or `license_id` is null.
- Removes duplicate rows.

9.3 Merge Logic

The cleaned license DataFrame is left joined to the cleaned owner DataFrame on:

<code>account_number</code>

A left join is used because the license records are the main transactional records. This keeps license records even when owner information is missing or unavailable.

For columns that exist in both DataFrames, the merge logic uses `coalesce` to prefer non-null values. Columns unique to each DataFrame are preserved in the merged output.

10 Output Files

The transformation step writes five Parquet outputs into the `parquet/` directory.

Output Folder	Description
<code>parquet/original_business_owners</code>	Raw Business Owners data as read by Spark.
<code>parquet/original_business_licenses</code>	Raw Business Licenses data as read by Spark.
<code>parquet/business_owners</code>	Cleaned Business Owners data.
<code>parquet/business_licenses</code>	Cleaned Business Licenses data.
<code>parquet/business_license_owners</code>	Merged license and owner dataset.

All outputs are written in overwrite mode using Snappy compression.

11 Testing Strategy

This section describes how I would test the correctness and reliability of the pipeline.

11.1 Download Testing

Download testing should verify that both CSV files are successfully downloaded before transformation begins.

Tests should check:

- The Business Owners CSV file exists after download.
- The Business Licenses CSV file exists after download.
- Downloaded files are not empty.

- Missing environment variables raise clear errors.
- Invalid URLs raise download failures.
- Temporary `.part` files are removed after failed downloads.
- Retry logic runs when a request fails.

11.2 Transformation Testing

Transformation testing should verify that the cleaning logic produces expected results.

Tests should check:

- Column names are converted to lowercase with underscores.
- String columns are trimmed.
- Owner names are formatted consistently.
- `full_name` is created correctly.
- Date columns are converted to date types.
- Numeric columns are converted to integer or double types where expected.
- Rows with null `account_number` or null `license_id` are removed from the cleaned license data.
- Duplicate rows are removed.
- The merged dataset keeps license records through the left join.

11.3 Validation Testing

Validation testing should verify that the validation module catches bad output.

Tests should check:

- Missing Parquet folders raise `FileNotFoundError`.
- Empty Parquet outputs raise validation errors.
- Missing required columns raise validation errors.
- Null values in important fields such as `account_number` and `license_id` raise validation errors.
- Valid Parquet outputs return a successful validation summary.

11.4 Reliability Testing

Reliability testing should verify that the pipeline can handle failures safely.

Tests should check:

- The pipeline can be run multiple times without creating duplicate outputs.
- Failed downloads do not leave incomplete CSV files as final outputs.
- Spark read failures stop the pipeline before cleanup.
- Cleanup only runs after validation succeeds.
- Airflow task dependencies prevent transformation from running before both downloads finish.
- Airflow does not run multiple overlapping pipeline executions because `max_active_runs=1`.

11.5 Data Correctness Testing

Data correctness testing should verify that the final dataset makes sense analytically.

Tests should check:

- The merged dataset contains `account_number` and `license_id`.
- The merged dataset row count is greater than zero.
- The merged dataset row count is consistent with the cleaned license dataset because licenses are used as the left side of the join.
- Known account numbers from both datasets successfully join.
- Geographic fields such as ward, community area, latitude, and longitude remain available for analysis.
- Owner fields such as `full_name`, owner title, and legal entity owner are available where matching owner records exist.

12 Conclusion

This pipeline creates a repeatable process for collecting, cleaning, merging, validating, and storing Chicago business license and ownership data. The final merged dataset connects license activity with ownership information, allowing users to analyze who owns licensed businesses, where businesses operate, what license statuses exist, and how business activity is distributed across Chicago.